

Ponto de extensão no *metadata service*

O ponto de extensão está no método `get_preseed()` do `AnonMetaDataHandler`, em `metadataserver/api.py`.

Quando a máquina está em estado `DEPLOYING`, o método verifica a existência da chave `multi_boot_preseed` no `NodeMetadata`:

```
def get_preseed(self, request, version=None, system_id=None):
    node = get_object_or_404(Node, system_id=system_id)

    if node.status == NODE_STATUS.DEPLOYING:
        multi_boot = NodeMetadata.objects.filter(
            node=node, key="multi_boot_preseed"
        ).first()

    if multi_boot:
        # Gera o cloud-config completo (datasource, rsyslog,
        # reporting) e injeta o orquestrador
        token = NodeKey.objects.get_token_for_node(node)
        metadata_url = build_metadata_url(
            request, route, node,
            node.get_boot_rack_controller(),
        )
        cloud_config = get_base_preseed(node)
        cloud_config.update({
            "datasource": {
                "MAAS": {
                    "metadata_url": metadata_url,
                    "consumer_key": token.consumer.key,
                    "token_key": token.key,
                    "token_secret": token.secret,
                }
            },
            "rsyslog": {
                "remotes": {
                    "maas": get_rsyslog_host_port(request, node)
                }
            },
        })
        cloud_config.update(
            get_cloud_init_reporting(request, node, token)
        )

        cloud_config["write_files"] = [
            {
                "path": "/tmp/multi-boot.sh",
                "content": multi_boot.value,
                "permissions": "0755",
            }
        ]
        cloud_config["runcmd"] = ["/tmp/multi-boot.sh"]
        cloud_config["bootcmd"] = [
            ["sh", "-c",
             "echo '[multiboot] started at $(date)' > /dev/kmsg"]
        ]
```

```
    return HttpResponse(
        ("#cloud-config\n" +
         yaml.safe_dump(cloud_config)).encode("utf-8"),
        content_type="text/plain",
    )

# Fluxo padrão
preseed = get_preseed(request, node)
return HttpResponse(preseed, content_type="text/plain")
```